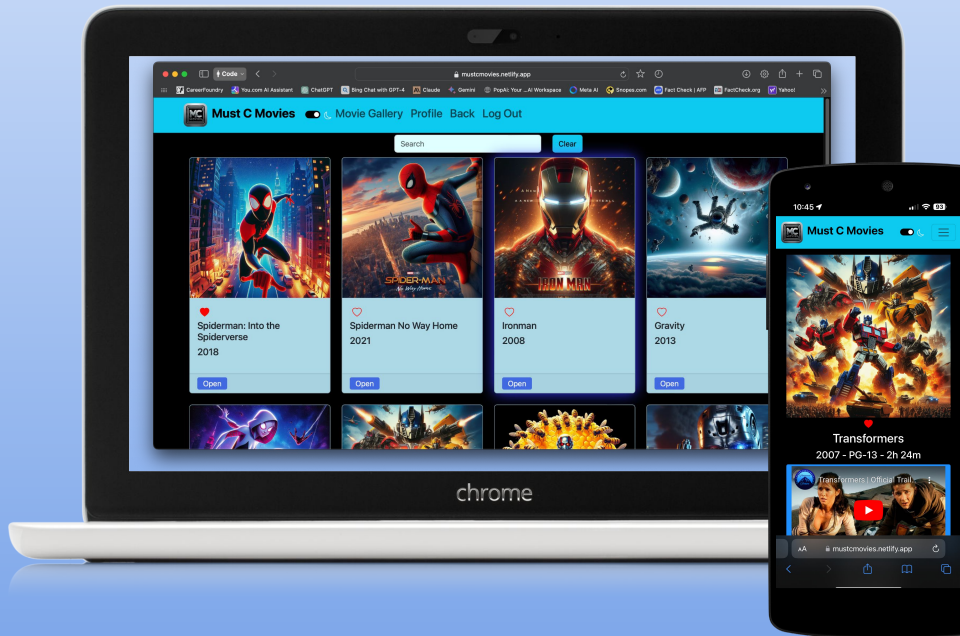




Must C Movies Web App



Overview

Must C Movies

- Innovative web app developed with the cutting-edge MERN stack.
- Providing user access to info on movies, directors, and genres.
- User Account creation.
- Personal data management.
- Curating personalized lists of favorite movies.



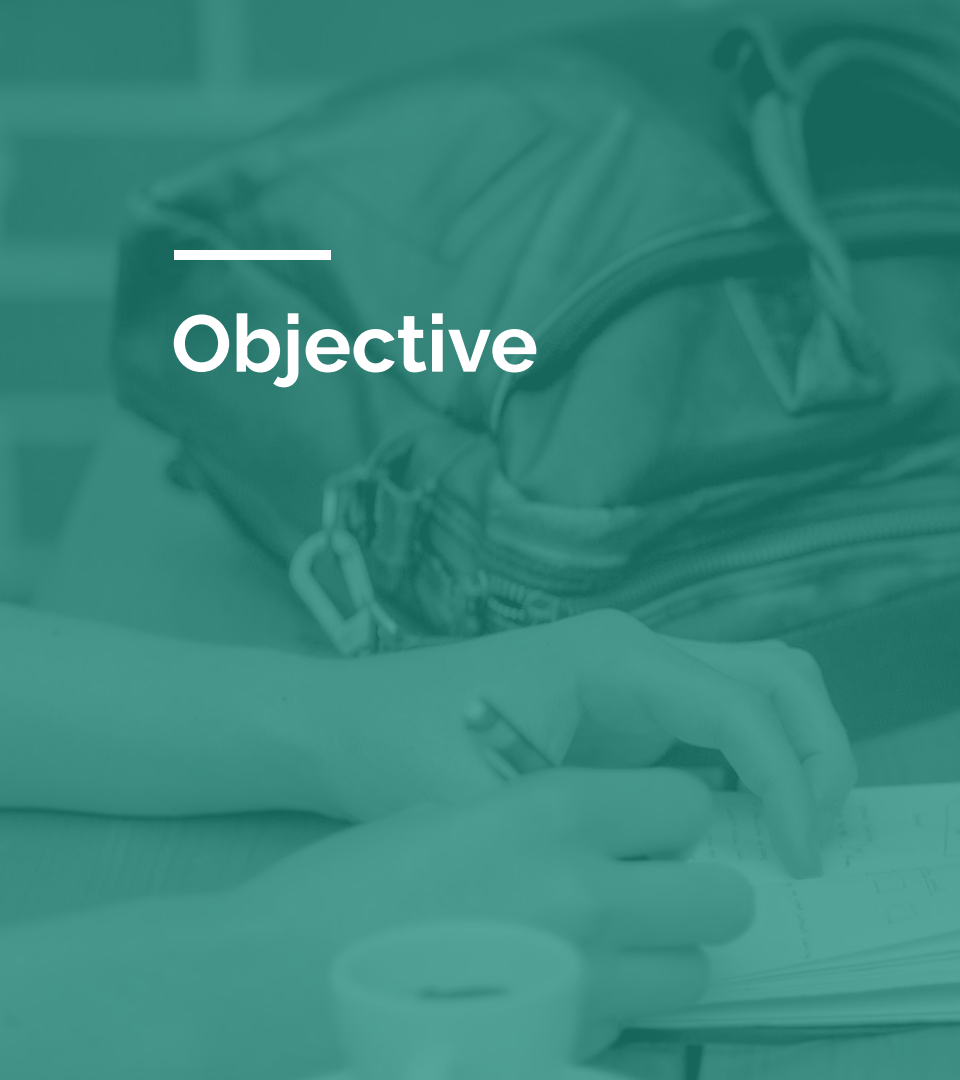
Demo Web App

The Purpose

- A dynamic web application built with the MERN stack.
- Offers a comprehensive database of movies, directors, and genres.
- Features user-friendly functionalities, including:

Account creation.

- Personal data management.
- Creation of personalized lists of favorite movies.



Objective

Built a full-stack development project to demonstrate a dynamic web app from scratch, seamlessly integrating server side and client side functionalities for a professional portfolio.



Methods

- **Front End: Deployed via Netlify**
- **React Libraries:**
 - **React Bootstrap**
 - **React Router Dom**
 - **React Icons**
 - **Prop Types**
 - **React Cool On Click Outside**
- **Backend: Deployed via Heroku**
 - **Mongoose (ODM - Object Data Modeling)**
 - **Body Parser**
 - **Lodash**
- **Authentication:**
 - **JWT (Json Web Tokens)**
 - **Passport**
 - **Cors**
 - **Bcrypt**
- **Middleware:**
 - **Morgan (For Logging)**

Challenges: Part 1

- After Countless Trial & Error: **Backend Challenges** I had to overcome
 - Lack of visual feedback posed a significant hurdle compared to frontend development.
 - Relied on tools like Postman and strategic Console Log implementations for insight into code outcomes and error troubleshooting.
 - Leveraged resources such as Chrome Developer Web Inspect Console and Heroku logs to trace error origins and enhance understanding of backend architecture.
 - Shifted approach to troubleshooting by isolating issues instead of scanning the entire codebase.
-

Challenges: Part 2

- **Front End Challenges**

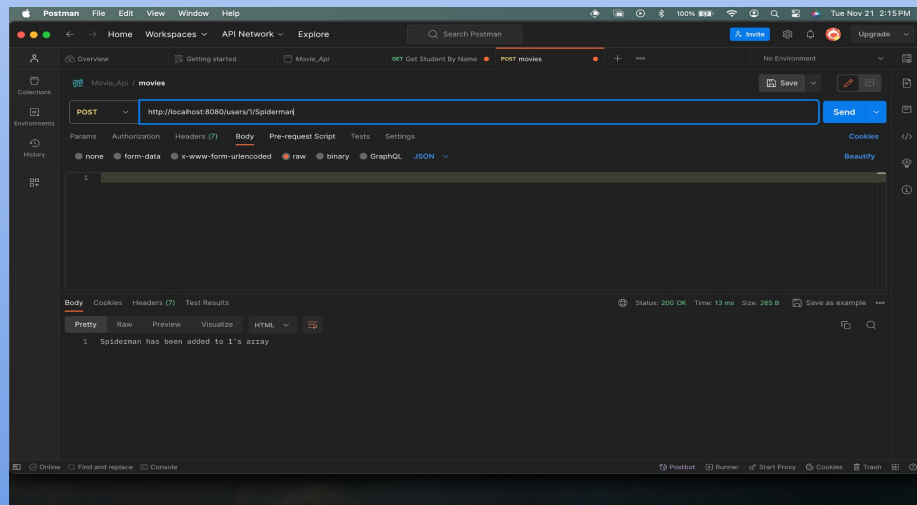
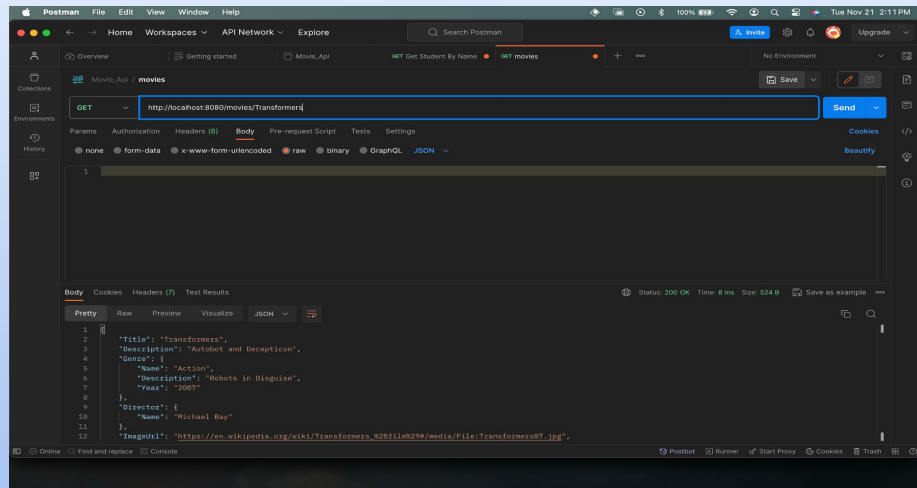
- Error messages lacked clarity and were scattered, making troubleshooting more challenging compared to the backend.
- Therefore Applying Web Inspect Console Log Techniques to help isolate issue.
- Component Interdependencies:

Making changes or resolving issues in one component could inadvertently impact or break other components due to dependencies and prop originations, requiring a deeper understanding of the application's structure and interconnections.

Server-Side

API Development:

- The Movie API Database was initially constructed using Postgresql but later migrated to MongoDB for enhanced functionality and scalability implementing Model Business Logic.
- Created a RESTful API using Node.js and Express.
- Interacts with a non-relational MongoDB database.
- Utilizes standard HTTP methods like GET and POST.
- Implements CRUD operations for data retrieval and storage.
- Delivers movie information in JSON format for seamless integration and accessibility

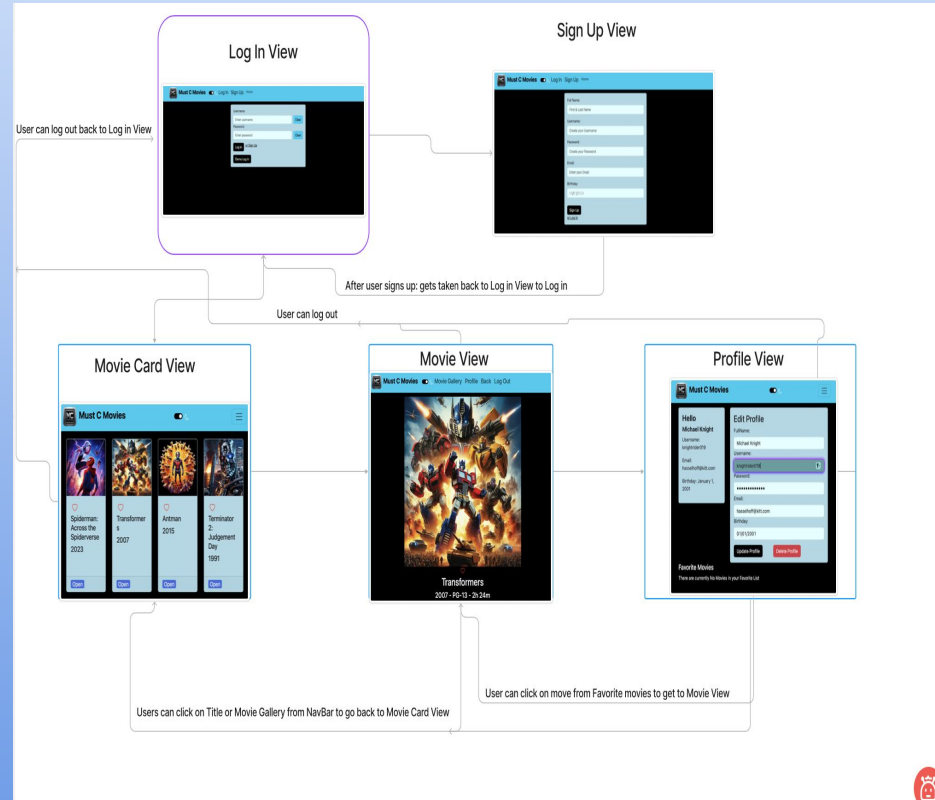


Client Side

User Interface Development:

- Crafted a single-page, responsive application using React.
- Various interface views include:
 - Main view: Displays a comprehensive list of movies.
 - Single movie view: Provides detailed movie information and allows adding to favorites.
 - Login view.
 - Registration view.
 - Profile view: Enables users to update personal data and manage favorite movies list

User Flow



Further Features



- Enhanced Security Measures:
 - Authentication Approach:
 - Leveraged basic HTTP for initial login.
 - Implemented JWT token-based authorization to fortify site security.
 - Security Features:
 - Incorporated CORS for enhanced security.
 - Employed password hashing to safeguard user credentials.
 - Implemented rigorous data validation protocols to ensure data integrity and further fortify site security.

Discovery

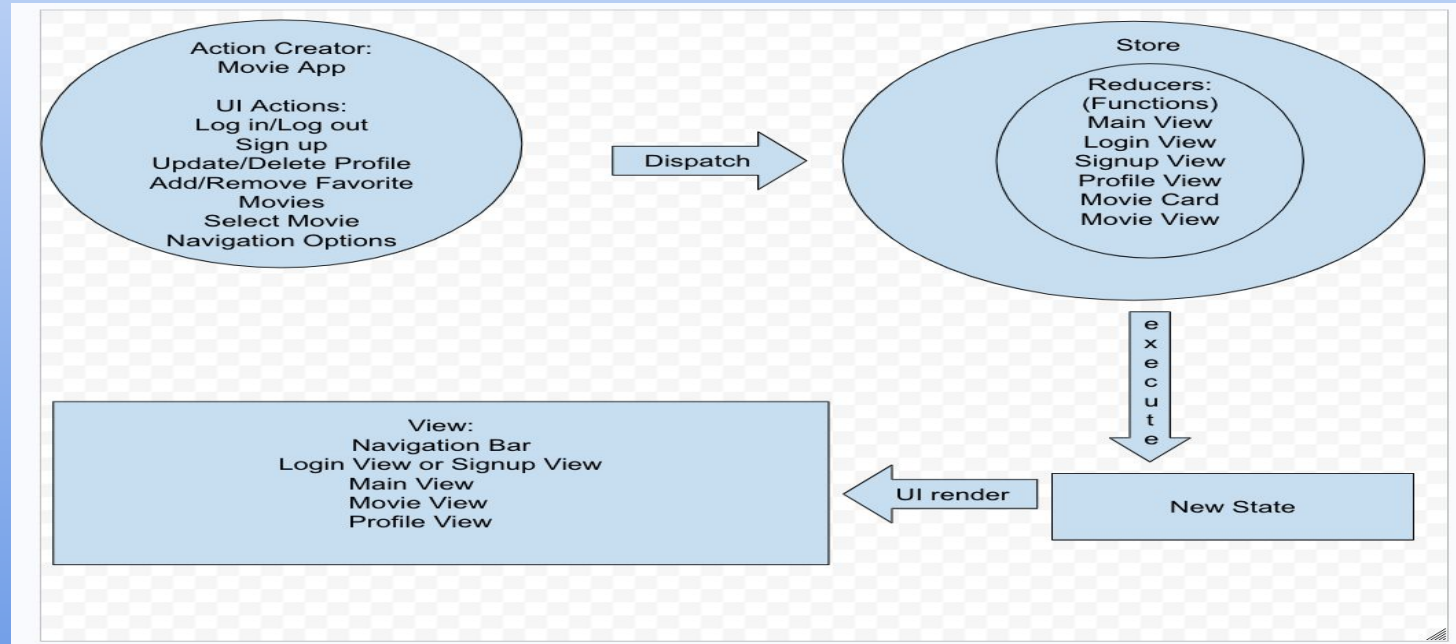
- Security Decisions:
 - Disabled the 'get all users' feature to prevent unauthorized access to private information between users..
- Additional Decisions:
 - Embedded Movie Trailers
 - Added Special Glow Effects on Hover
 - Added Confirm Alert when Delete Profile
 - Similar Movie Sort & Search Bar



Use cases / user stories

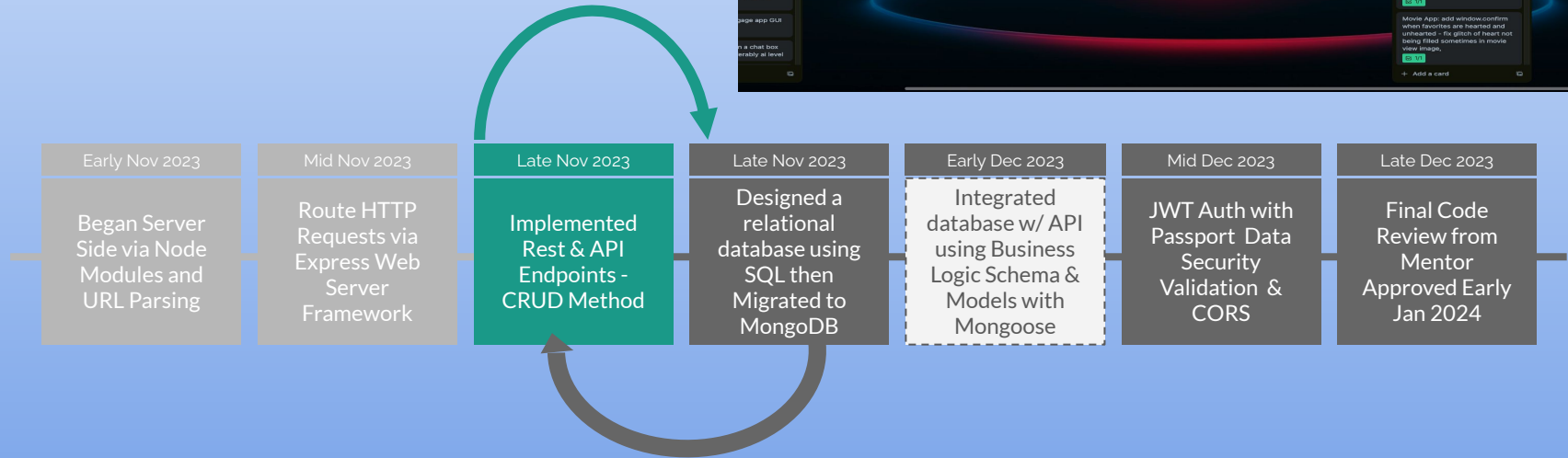
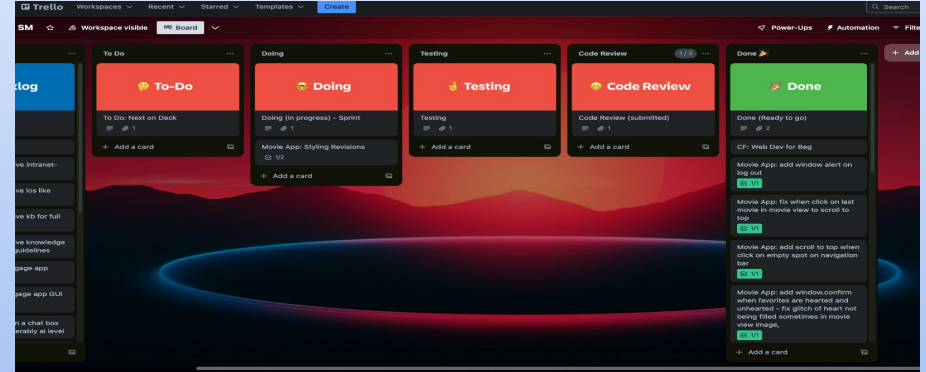
- “As an user, I want to be able to receive info on movies, directors & genres so that I can learn more about movies I’ve watched or am interested in.”
- “As a user, I want to be able to create a profile so I can save data about my favorite movies.”

Future Flux Flow Rendition



Trello Project Workflow Management:

Timeline





Future Upgrades

- Angular (Mean Stack) & Redux versions
- Add Admin Dashboard for Admin Access
- Future Features:
 - Forgot Password
 - Google Auth Login integration
 - Trailer Playlist AKA Player

Credits

Role: Sam My (Lead Developer)

CF Tutor: James Katarikawe

CF Mentors: Sarah Rohl & Leno (Colleen) Gillis

Honorable Mention:

Shot out to my Study Buddy: AJ Sorbello

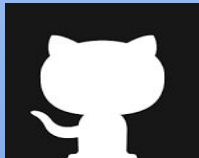
Special Thanks

To Team:



*CAREER***FOUNDRY**

Client-Side
Github Link



Server-Side
Github Link



Movie App
Demo Link

